

CAIO JUN TAKAHASHI, 12488380

**REPRODUÇÃO DA ARQUITETURA DO GAME BOY
ADVANCE EM FPGA**

São Paulo
2026

CAIO JUN TAKAHASHI, 12488380

**REPRODUÇÃO DA ARQUITETURA DO GAME BOY
ADVANCE EM FPGA**

Trabalho apresentado à Escola Politécnica da
Universidade de São Paulo para obtenção do
Título de Engenheiro Engenharia Elétrica com
ênfase em Sistemas Eletrônicos.

São Paulo
2026

CAIO JUN TAKAHASHI, 12488380

**REPRODUÇÃO DA ARQUITETURA DO GAME BOY
ADVANCE EM FPGA**

Trabalho apresentado à Escola Politécnica da
Universidade de São Paulo para obtenção do
Título de Engenheiro Engenharia Elétrica com
ênfase em Sistemas Eletrônicos.

Área de Concentração:

Sistemas Eletrônicos

Orientador:

Prof. Dr. Bruno Cavalcante de Souza
Sanches

São Paulo
2026

RESUMO

Este trabalho propõe a reprodução da arquitetura do console portátil Game Boy Advance (GBA) em uma plataforma de hardware reconfigurável do tipo FPGA. O objetivo é descrever em *Verilog* os principais blocos funcionais do console, o núcleo de processamento ARM7TDMI, o subsistema de memória (BIOS, EWRAM, IWRAM, VRAM, OAM e Palette RAM), o controlador de barramento e, em etapa posterior, a unidade de processamento gráfico e a interface de saída de vídeo, de modo que o sistema integrado seja capaz de executar programas binários originalmente compilados para o GBA. A implementação é validada na placa Altera DE1-SoC, que contém uma FPGA Cyclone V, utilizando o fluxo de projeto Quartus Prime, simulação no ModelSim/Questasim e *toolchain* arm-none-eabi para a geração de programas de teste em *assembly* ARM e Thumb. Diferentemente da emulação puramente em software, a abordagem proposta procura preservar a temporização e o comportamento original do hardware, o que confere maior fidelidade à reprodução e, ao mesmo tempo, permite um estudo aprofundado de arquitetura de computadores e de projeto de sistemas digitais. No estágio atual do desenvolvimento, correspondente à presente entrega intermediária, o núcleo ARM7TDMI já se encontra implementado e validado, tanto em simulação quanto em hardware, e constitui a prova de conceito apresentada neste momento. O subsistema de memória, mapa de memória do GBA, controlador e árbitro de barramento, canal de DMA e controlador de SDRAM para a ROM de cartucho, já se encontra descrito e integrado ao *datapath*, com a validação funcional do caminho integrado ainda em andamento; o subsistema de vídeo e a integração final permanecem como continuidade prevista do trabalho.

Palavras-Chave, FPGA, Game Boy Advance, ARM7TDMI, Arquitetura de Computadores, Verilog, Sistemas Digitais Reconfiguráveis.

ABSTRACT

This work proposes the reproduction of the Game Boy Advance (GBA) portable console architecture on a reconfigurable FPGA platform. The goal is to describe in *Verilog* the main functional blocks of the console, the ARM7TDMI processing core, the memory subsystem (BIOS, EWRAM, IWRAM, VRAM, OAM and Palette RAM), the bus controller and, in a later stage, the graphics processing unit and the video output interface, so that the integrated system is able to execute binary programs originally compiled for the GBA. The implementation is validated on the Altera DE1-SoC board, which carries a Cyclone V FPGA, using the Quartus Prime design flow, ModelSim/Questasim simulation and the arm-none-eabi toolchain to produce ARM and Thumb assembly test programs. Differently from purely software emulation, the proposed approach aims to preserve the original timing and behaviour of the hardware, which provides a higher fidelity reproduction and, at the same time, allows a deeper study of computer architecture and digital system design. At the current development stage, corresponding to this intermediate delivery, the ARM7TDMI core is already implemented and validated, both in simulation and on hardware, and constitutes the proof of concept presented at this point. The memory subsystem, the GBA memory map, the bus controller and arbiter, a DMA channel and an SDRAM controller for the cartridge ROM, is already described and integrated into the datapath, with functional validation of the integrated path still ongoing; the video subsystem and the final integration remain the planned continuation of the work.

Keywords, FPGA, Game Boy Advance, ARM7TDMI, Computer Architecture, Verilog, Reconfigurable Digital Systems.

LISTA DE FIGURAS

- 1 Diagrama de blocos interno do núcleo ARM7TDMI, com o *datapath* (registrador de endereços, incrementador, banco de registradores, multiplicador, *barrel shifter* e ALU de 32 bits) e a unidade de decodificação e controle, à direita. Fonte: ARM Limited [1]. 7

- 2 Arquitetura interna do Game Boy Advance (*die* CPU AGB): núcleo ARM7TDMI, co-processador Sharp SM83 (retrocompatibilidade), as regiões de memória (WRAM interna e externa, VRAM), a PPU, o gerador de som, o controlador de DMA e a interface com o *Game Pak*; indicam-se as larguras de barramento de cada bloco. Fonte: Copetti [3]. 8

- 3 *Netlist* RTL do projeto no estágio atual, extraída pelo *RTL Viewer* do Quartus Prime após a análise e síntese. Reúne o núcleo ARM7TDMI já implementado e validado e os blocos do subsistema de memória ainda em integração (*bus_controller*, módulos de região e controlador de SDRAM). Fonte: autoria própria. 13

- 4 Arquitetura-objetivo do projeto. O núcleo ARM7TDMI (azul) já se encontra implementado e validado; o subsistema de memória (laranja), *bus_controller*, regiões internas (BIOS, EWRAM, IWRAM, VRAM, I/O, Palette, OAM), PAK_RAM e o controlador da SDRAM externa, que abriga exclusivamente a PAK_ROM, além da arbitragem de barramento e do DMA, encontra-se implementado e integrado ao *datapath*, com a validação funcional do caminho integrado ainda em andamento; os blocos de PPU e saída de vídeo, gerador de som e áudio, e a interface de entrada (*Key Controller*) permanecem planejados (cinza). As larguras indicadas correspondem ao barramento nominal de cada região; a decodificação por região emprega o *nibble* alto do endereço (*addr* [27:24]). 16

LISTA DE SIGLAS

ALU	Arithmetic Logic Unit
ARM	Advanced RISC Machines
ASIC	Application-Specific Integrated Circuit
BIOS	Basic Input/Output System
CPSR	Current Program Status Register
CPU	Central Processing Unit
DMA	Direct Memory Access
EWRAM	External Work RAM
FPGA	Field-Programmable Gate Array
GBA	Game Boy Advance
GPU	Graphics Processing Unit
HDL	Hardware Description Language
IRQ	Interrupt Request
IWRAM	Internal Work RAM
OAM	Object Attribute Memory
PC	Program Counter
PLL	Phase-Locked Loop
PPU	Picture Processing Unit
RAM	Random Access Memory
ROM	Read-Only Memory
RTL	Register Transfer Level
SoC	System-on-Chip
SPSR	Saved Program Status Register
SRAM	Static Random Access Memory
SDRAM	Synchronous Dynamic Random Access Memory
VGA	Video Graphics Array
VRAM	Video RAM

SUMÁRIO

1	Introdução	1
1.1	Antecedentes	1
1.2	Motivação	2
1.3	Relevância	2
1.4	Declaração do Problema	2
1.5	Declaração da Necessidade	3
1.6	Requisitos de Marketing	3
1.7	Requisitos de Engenharia	4
1.8	Árvore de Objetivos	4
2	Estado da Arte	6
2.1	Conceitos e Definições	6
2.1.1	Hardware reconfigurável e FPGAs	6
2.1.2	Linguagens de descrição de hardware	6
2.1.3	Arquitetura ARM7TDMI	6
2.1.4	Arquitetura do Game Boy Advance	7
2.2	Abordagem Teórica	8
2.3	Soluções Existentes	8
2.4	Tendências	9
2.5	Conclusão do Capítulo	9
3	Materiais e Métodos	11
3.1	Plataforma de Implementação	11
3.2	Fluxo de Projeto e Ferramentas	11

3.3	Descrição da Prova de Conceito	12
3.3.1	Camada 1: Núcleo ARM7TDMI (implementada e validada)	12
3.3.2	Camada 2: Subsistema de Memória (integrada, em validação)	13
3.3.3	Camada 3: Saída de Vídeo (planejada)	14
3.3.4	Camada 4: Integração e Validação (planejada)	14
3.4	Metodologia de Verificação	14
3.5	Cronograma Macro	15
Referências		17

1 INTRODUÇÃO

1.1 Antecedentes

O Game Boy Advance (GBA), lançado pela Nintendo em 2001, foi o sucessor direto do Game Boy Color e representou um salto arquitetural relevante na linha de consoles portáteis da empresa. Diferentemente de seus predecessores, baseados em variações do Z80, o GBA adota como núcleo principal o processador ARM7TDMI, um *core* RISC de 32 bits desenvolvido pela ARM Limited e largamente utilizado em sistemas embarcados da primeira metade da década de 2000 [1,2]. Em complemento, o console preserva um co-processador Sharp LR35902 (compatível com o Game Boy Color) para retrocompatibilidade com a biblioteca anterior, bem como blocos dedicados a vídeo, áudio e acesso direto à memória [3,4].

A arquitetura do GBA combina, portanto, um processador de uso geral, um espaço de endereçamento segmentado em múltiplas regiões com características distintas de largura, latência e função (BIOS, EWRAM, IWRAM, registradores de I/O, Palette RAM, VRAM, OAM, ROM de cartucho e SRAM de salvamento) e uma PPU capaz de operar em seis modos gráficos distintos, alguns baseados em *tilemaps* e outros em *bitmaps* [3–5]. Esse conjunto torna o console uma plataforma compacta, porém arquiteturalmente rica, e didaticamente interessante para o estudo de organização de computadores e projeto de sistemas digitais.

Do ponto de vista tecnológico, as últimas duas décadas viram o amadurecimento das FPGAs como veículo de reprodução fiel de arquiteturas históricas. Diferentemente da emulação por software, em que o comportamento do hardware original é *imitado* por um programa executado sobre um processador moderno, a implementação em FPGA descreve, em linguagem de descrição de hardware (HDL), os blocos digitais que compõem o sistema, sintetizando-os em uma malha de células lógicas reconfiguráveis [6, 7]. O resultado é um sistema digital efetivamente equivalente ao original do ponto de vista da temporização e do comportamento ciclo a ciclo, ainda que materializado em um *die* distinto.

1.2 Motivação

A motivação para o desenvolvimento deste trabalho parte de três vetores convergentes. O primeiro é a **preservação histórica**: o GBA é um console descontinuado desde 2010, cujo *die* original (*Custom CPU Agb*) não é mais fabricado e cujos exemplares em bom estado de conservação se tornam progressivamente escassos e caros no mercado de usados [3]. O segundo vetor é **didático**: a reprodução em hardware reconfigurável da arquitetura do console exige a aplicação integrada de conhecimentos de arquitetura de computadores, projeto digital, verificação funcional, ferramentas de EDA e *toolchains* de *cross-compilation*, o que o torna um projeto de Conclusão de Curso de elevado retorno formativo. O terceiro vetor é **tecnológico**: trabalhos como MiSTer FPGA [11] e Analogue Pocket [12] demonstraram que FPGAs de custo acessível são hoje capazes de hospedar reproduções completas de consoles da era 8/16/32 bits, configurando uma fronteira ativa e viável de pesquisa aplicada.

1.3 Relevância

A relevância deste trabalho manifesta-se em três frentes complementares. Do ponto de vista **acadêmico**, o projeto percorre, de ponta a ponta, o ciclo de vida de um sistema digital não trivial: parte da especificação extraída da documentação técnica original do processador [1,2] e do console [3,5], passa pela descrição RTL em *Verilog* e culmina na verificação por simulação e na síntese física em FPGA real. Do ponto de vista **preservacionista**, e ao contrário de uma emulação em software, a implementação em FPGA preserva diretamente o *comportamento* do hardware original, sua estrutura de barramento, sua hierarquia de memória e suas relações de temporização [7], contribuindo para a documentação viva de uma arquitetura que tende a se perder com o tempo. Por fim, do ponto de vista **tecnológico**, o desenvolvimento abre caminho para extensões futuras, como a inclusão de interface de cartucho físico, áudio e controle externo, ou a portabilidade para FPGAs de menor custo, demonstrando que projetos de nicho podem ser realizados com os recursos didáticos disponíveis em ambiente universitário.

1.4 Declaração do Problema

A execução fiel de jogos originalmente publicados para o GBA depende, hoje, ou (i) da disponibilidade do console físico original, cuja oferta no mercado é decrescente e cujo custo é crescente; ou (ii) de emuladores em software que, embora maduros, executam-se sobre processadores de propósito geral e introduzem desvios de temporização e comportamento difíceis

de auditar [3, 13]. Em ambos os casos, o usuário final não dispõe de uma plataforma de hardware aberta, de baixo custo e fidelidade arquitetural, capaz de servir simultaneamente para uso pessoal, para fins de preservação e para fins didáticos.

O problema central deste trabalho é, portanto, formulado como:

Como reproduzir, em uma plataforma de hardware reconfigurável de baixo custo, a arquitetura do Game Boy Advance, de modo a permitir a execução de programas binários originalmente compilados para o console, preservando seu comportamento ao nível de barramento e de transferência de registradores?

1.5 Declaração da Necessidade

Decorre desse problema a necessidade de uma plataforma de hardware que reúna um conjunto bem definido de características. Em primeiro lugar, ela deve executar, de forma funcionalmente correta, o conjunto de instruções ARMv4T (ARM e Thumb) tal como implementado no núcleo ARM7TDMI [1]. Deve, ainda, respeitar o mapa de memória do GBA, com suas regiões de BIOS, EWRAM, IWRAM, registradores de I/O, Palette RAM, VRAM, OAM, ROM de cartucho e SRAM de *save*, observando a respectiva largura nominal de barramento [3, 4]. Para permitir a visualização da execução, a plataforma precisa oferecer uma interface de saída de vídeo compatível com monitores de uso comum, VGA, no presente trabalho. Tudo isso deve ser obtido sobre recursos de hardware disponíveis em uma placa didática de uso corrente (Altera DE1-SoC, Cyclone V), sem depender de componentes custosos ou descontinuados [9, 10]. Por fim, o projeto deve apoiar-se em um fluxo aberto, baseado em Quartus Prime e ModelSim/QuestaSim, de modo a permitir sua reprodução, auditoria e extensão por terceiros.

1.6 Requisitos de Marketing

Considerando o público-alvo do projeto, entusiastas de retrocomputação, estudantes de engenharia e laboratórios didáticos de projeto digital, identificaram-se cinco requisitos de *marketing*. O primeiro deles (M1) é reproduzir, na medida do possível, a experiência de uso do console original, incluindo a execução de jogos, a resposta ao *joypad* e a saída de vídeo e áudio. A esse soma-se a exigência de um custo total de implementação compatível com o orçamento de um trabalho acadêmico de graduação (M2) e a opção por uma plataforma didática já consolidada no ensino de engenharia, de modo a maximizar a reprodutibilidade do projeto (M3). Pretende-se, ainda, que o resultado seja *open-source*, com código, documentação e cronograma

publicamente disponíveis (M4), e que apresente uma barreira de entrada compatível com o nível de um aluno de graduação familiarizado com lógica digital e linguagem Verilog (M5).

1.7 Requisitos de Engenharia

Os requisitos de *marketing* traduzem-se em um conjunto de requisitos de engenharia, de natureza mensurável e verificável. No que diz respeito ao processamento, o projeto deve implementar em Verilog um *core* compatível com a especificação ARMv4T, modos ARM e Thumb, com suporte aos modos de operação Usuário, Sistema, Supervisor, Abort, Undefined, IRQ e FIQ e seus bancos de registradores devidamente espelhados [1] (E1), bem como uma unidade de decodificação multi-ciclo capaz de sequenciar instruções de transferência de dados, de transferência em bloco, multiplicação e desvios condicionais, atendendo às latências previstas na especificação (E2). O subsistema de memória requer um controlador de barramento que decodifique os bits [27:24] do endereço e roteie cada transação à região correta do mapa, com larguras de palavra de 8, 16 e 32 bits e tratamento de alinhamento (E3), além da integração com a SDRAM da placa DE1-SoC para abrigar a ROM de cartucho (PAK_ROM) por meio de um controlador dedicado [9], ficando as demais regiões em memória *on-chip* (E4). A saída gráfica demanda a geração de um sinal VGA padrão, por exemplo, 640×480 a 60 Hz, mapeado a partir do *framebuffer* de 240×160 do GBA, com seus sincronismos horizontal e vertical (E5). No plano do controle de fluxo, é necessário o tratamento das exceções SWI, Undefined, Prefetch Abort, Data Abort, IRQ e FIQ, com transição correta de modo e salvamento de PC em LR e de CPSR em SPSR (E6). A verificabilidade do sistema exige um ambiente de simulação no ModelSim/QuartaSim capaz de carregar imagens binárias geradas pela toolchain GNU (*arm-none-eabi-as*, *-ld* e *-objcopy*) e de inspecionar o *datapath* via *SignalTap* na FPGA real (E7). Há, por fim, duas restrições físicas: um *slack* temporal positivo em todas as malhas síncronas, partindo de uma frequência de operação reduzida (de centenas de Hz a poucos MHz), com folga para o escalonamento progressivo rumo à frequência nominal do GBA, de 16,78 MHz para o ARM7TDMI [3] (E8); e um consumo de recursos da Cyclone V (ALMs, M10K e DSPs) compatível com a capacidade do dispositivo 5CSEMA5F31C6 [9, 10] (E9).

1.8 Árvore de Objetivos

A meta global do projeto (O0), reproduzir a arquitetura do Game Boy Advance em FPGA, de modo a executar binários nativos do console, decompõe-se em quatro objetivos específicos. O primeiro (O1) é implementar um núcleo ARM7TDMI sintetizável em Verilog, o que abrange

o banco de registradores com cópias espelhadas por modo de operação, a ALU com geração das *flags* NZCV, o *barrel shifter* integrado ao operando B, o multiplicador 32×32 com saídas *Lo/Hi*, a unidade de decodificação multi-ciclo cobrindo ARM e Thumb e o tratamento completo de exceções (Reset, SWI, Undef, Prefetch Abort, Data Abort, IRQ e FIQ). O segundo objetivo (O2) é implementar o subsistema de memória do GBA, com os modelos sintetizáveis das regiões internas (BIOS, EWRAM, IWRAM, I/O, Palette e OAM), o controlador de SDRAM para a ROM de cartucho externa (PAK_ROM), o controlador de barramento com decodificação por região e *mux* de leitura, o árbitro de barramento entre o processador e os canais de DMA, e o tratamento dos tamanhos de acesso (*byte*, *halfword* e *word*) com extensão de sinal. O terceiro (O3) consiste em implementar a saída de vídeo, incluindo a geração dos sinais VGA com sincronismos e *blanking*, o *framebuffer* 240×160 mapeado a partir da VRAM e o suporte inicial ao Modo 3 do GBA (*bitmap* 16 bpp), com extensão progressiva aos demais modos. O quarto e último objetivo (O4) é integrar e validar o sistema: executar programas de teste em ARM e Thumb gerados por *arm-none-eabi*, verificar funcionalmente o tratamento de interrupções e exceções, carregar uma imagem binária do tipo *homebrew* com exibição de vídeo em monitor e, enfim, documentar os resultados como comprovação da Prova de Conceito.

2 ESTADO DA ARTE

2.1 Conceitos e Definições

2.1.1 Hardware reconfigurável e FPGAs

Uma *FPGA* é um circuito integrado composto por uma malha de células lógicas programáveis, blocos de memória embarcada, blocos DSP e interconexões configuráveis, cuja função é definida em tempo de configuração por meio de um *bitstream* [6, 7]. Ao contrário de um ASIC, cuja função é imutável após a fabricação, uma FPGA pode ser reconfigurada em campo, o que a torna particularmente atrativa para prototipação de sistemas digitais, para reprodução de arquiteturas históricas e para aplicações em que o ciclo de *tape-out* é proibitivo [8].

2.1.2 Linguagens de descrição de hardware

A descrição funcional de um sistema digital sintetizável é tipicamente expressa em uma *Hardware Description Language* (HDL), Verilog, SystemVerilog ou VHDL, a partir da qual ferramentas de síntese inferem a rede de portas equivalente e ferramentas de *place-and-route* produzem o *bitstream* final [7]. Neste trabalho adota-se *Verilog*, por sua maturidade no fluxo Quartus Prime e por sua adequação à descrição RTL de microarquiteturas como a do ARM7TDMI.

2.1.3 Arquitetura ARM7TDMI

O ARM7TDMI é um processador RISC de 32 bits, com pipeline de três estágios (Fetch, Decode, Execute), suporte simultâneo aos conjuntos de instruções ARM (32 bits) e Thumb (16 bits), unidade de multiplicação *long* ($32 \times 32 \rightarrow 64$) e modelo de memória de Von Neumann [1, 2]. O conjunto de registradores visível ao usuário inclui $r0-r12$, *SP* ($r13$), *LR* ($r14$) e *PC* ($r15$), além de CPSR; um conjunto de bancos *shadow* de registradores e SPSRs é selecionado conforme o modo de operação (Usuário, Sistema, Supervisor, IRQ, FIQ, Abort e

Undefined) [1]. A Figura 1 apresenta o diagrama de blocos interno do núcleo, cujo *datapath*, registrador e incrementador de endereços, banco de registradores, multiplicador, *barrel shifter* e ALU de 32 bits, é reproduzido, módulo a módulo, na descrição RTL deste trabalho.

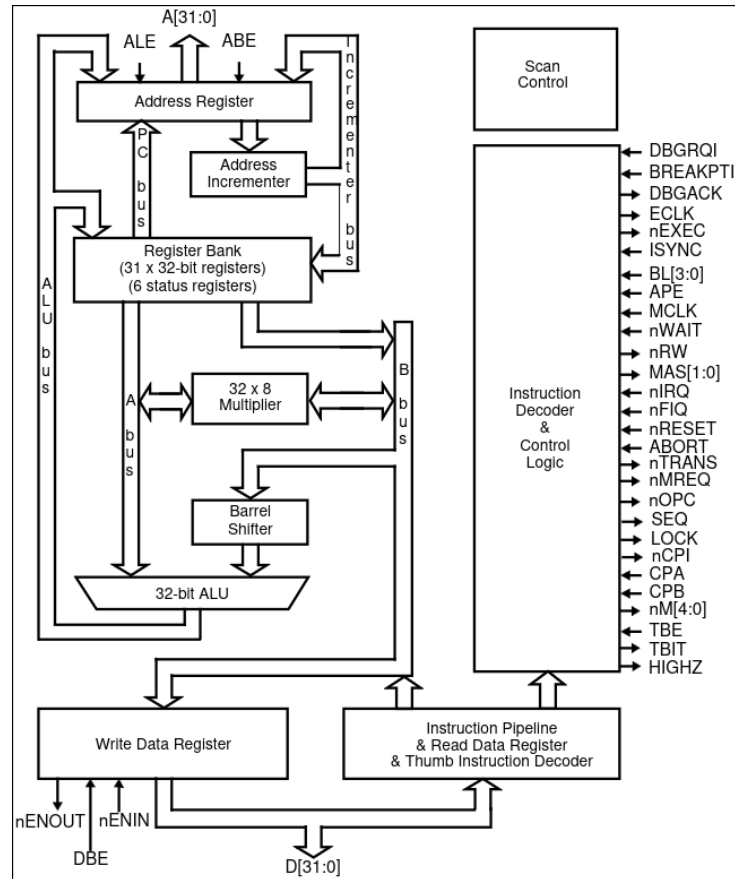


Figura 1: Diagrama de blocos interno do núcleo ARM7TDMI, com o *datapath* (registrador de endereços, incrementador, banco de registradores, multiplicador, *barrel shifter* e ALU de 32 bits) e a unidade de decodificação e controle, à direita. Fonte: ARM Limited [1].

2.1.4 Arquitetura do Game Boy Advance

O GBA é, em essência, um sistema-em-chip que combina o ARM7TDMI operando a 16,78 MHz, 96 KiB de VRAM, 32 KiB de IWRAM, 256 KiB de EWRAM, registradores de I/O memorizados em uma região dedicada 0x04000000, uma PPU com seis modos gráficos, quatro canais DMA e uma unidade de áudio com canais PSG e canais PCM [3–5]. A interface com o cartucho (*Game Pak*) ocupa a região 0x08000000-0x0DFFFFFF do mapa de memória, em barramento de 16 bits com modos de acesso configuráveis em latência [5]. Esses parâmetros estabelecem o conjunto-alvo a ser reproduzido neste trabalho. A Figura 2 resume essa organização interna, destacando os blocos funcionais e a largura nominal de barramento de cada região.

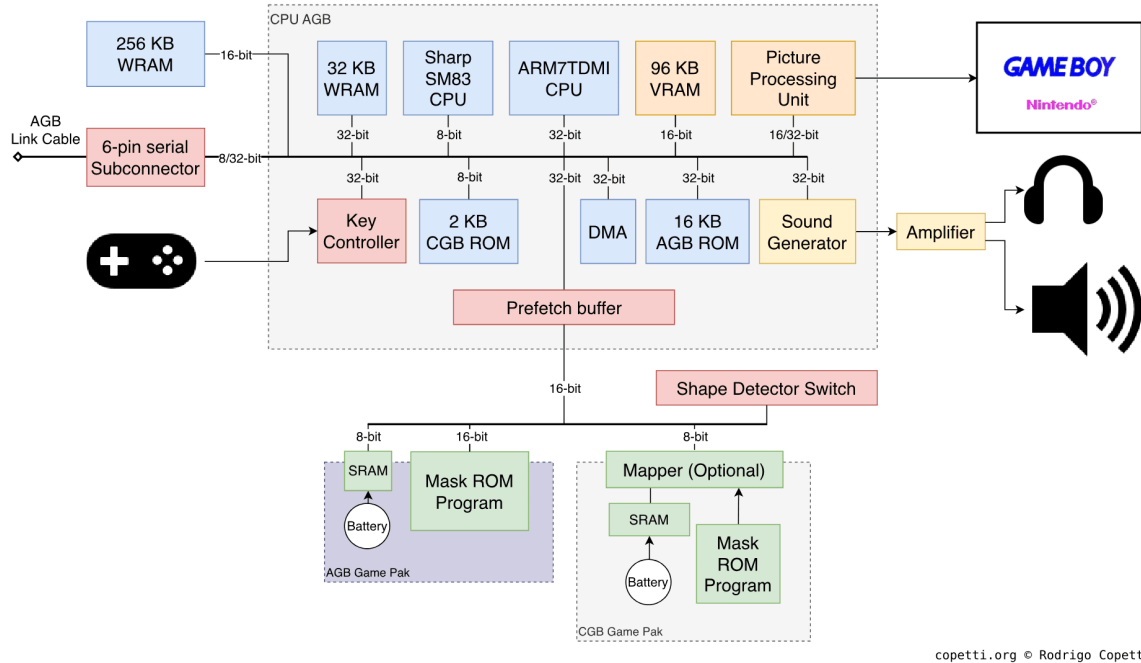


Figura 2: Arquitetura interna do Game Boy Advance (*die* CPU AGB): núcleo ARM7TDMI, co-processador Sharp SM83 (retrocompatibilidade), as regiões de memória (WRAM interna e externa, VRAM), a PPU, o gerador de som, o controlador de DMA e a interface com o *Game Pak*; indicam-se as larguras de barramento de cada bloco. Fonte: Copetti [3].

2.2 Abordagem Teórica

A reprodução de uma arquitetura legada em FPGA admite, em princípio, três níveis de fidelidade. No nível mais básico, a reprodução *funcional* (a) executa corretamente o conjunto de instruções e respeita o mapa de memória, mas mantém a temporização livre; é apropriada para fins didáticos e como ponto de partida da verificação. Um passo adiante, a reprodução *ciclo-aproximada* (b) acrescenta a essa funcionalidade a reprodução da contagem de ciclos de cada instrução conforme a especificação original. No nível mais exigente, a reprodução *ciclo-perfeita* (c) reproduz o comportamento ciclo a ciclo de todos os blocos, incluindo PPU, DMA e barramento, a ponto de tornar indistinguíveis, do ponto de vista do software, o original e a reprodução; é o padrão buscado por projetos como o MiSTer FPGA [11]. O presente trabalho almeja inicialmente a faixa (a)–(b), com uma arquitetura aberta à evolução incremental rumo a (c) em trabalhos futuros.

2.3 Soluções Existentes

A literatura técnica e a comunidade de *homebrew* oferecem diversas referências de reprodução do GBA, com diferentes compromissos de fidelidade e propósito. No campo da emulação em

software, o mGBA [14], escrito em C, destaca-se pela altíssima compatibilidade e pela ampla instrumentação para depuração, enquanto o NanoBoyAdvance [13], em C++, prioriza a precisão ciclo-aproximada e a legibilidade do código, o que o torna referência frequente para implementadores em hardware. No campo do hardware reconfigurável, o núcleo GBA do projeto MiSTer FPGA [11] persegue a alta fidelidade e a execução ciclo-perfeita sobre uma Cyclone V do tipo DE10-Nano, ao passo que o Analogue Pocket [12], produto comercial baseado em FPGA, reproduz Game Boy, Game Boy Color e Game Boy Advance e demonstra a viabilidade industrial da abordagem. Dão suporte a todas essas iniciativas a documentação de comunidade consolidada na engenharia reversa do hardware, com destaque para o GBATEK [5] e o Tonc [4], referência *de facto* para qualquer reprodução. No domínio acadêmico, projetos de reprodução de CPUs ARM em FPGA são bem documentados na literatura de ensino de arquitetura de computadores [6, 7], ainda que normalmente restritos a subconjuntos do ISA; a combinação de um ARM7TDMI completo com o subsistema gráfico do GBA permanece, contudo, um nicho explorado majoritariamente pela comunidade de *retrocomputing*.

2.4 Tendências

Três tendências balizam o estado da arte e o posicionamento deste trabalho. A primeira é o avanço do *open hardware* e a abertura de *toolchains*: a crescente disponibilidade de fluxos abertos para FPGA, como Yosys e nextpnr, para famílias selecionadas, e de plataformas didáticas a preços decrescentes amplia a base de pesquisadores capazes de reproduzir arquiteturas [8]. A segunda é a consolidação de *cores* reconfiguráveis colaborativos: projetos como o MiSTer [11] demonstram que reproduções ciclo-perfeitas podem ser construídas de forma incremental e mantidas por uma comunidade ampla. A terceira é a preservação digital: agências de preservação cultural reconhecem cada vez mais a importância de preservar não apenas o software, mas também o *comportamento* do hardware sobre o qual ele roda, e a reprodução em FPGA é o veículo técnico mais alinhado a esse objetivo [3].

2.5 Conclusão do Capítulo

O estado da arte mostra que a reprodução do GBA em FPGA é viável e desejável, com referências técnicas amplas (documentação ARM [1, 2], mapa de memória e periféricos [3–5]) e exemplos práticos consolidados [11, 12]. O espaço para contribuição original deste trabalho situa-se em duas frentes: a construção pedagógica de uma reprodução completa, com código aberto, documentado em português e desenvolvido em plataforma didática consolidada (DE1-

SoC); e a demonstração de que a graduação em Engenharia Elétrica oferece base suficiente para a execução de um projeto desta natureza.

3 MATERIAIS E MÉTODOS

Este capítulo descreve a plataforma, as ferramentas e o método de desenvolvimento e verificação adotados no projeto, bem como a Prova de Conceito que o materializa. Como a presente monografia acompanha uma **entrega intermediária** do Trabalho de Conclusão de Curso, a descrição da Prova de Conceito reflete o estágio atual do desenvolvimento: distingue-se, ao longo do texto, aquilo que já foi implementado e validado, e que constitui a demonstração apresentada neste momento, daquilo que se encontra em andamento ou planejado para a entrega final.

3.1 Plataforma de Implementação

A implementação é realizada na placa Altera DE1-SoC, da Terasic [10], que abriga uma FPGA Cyclone V (5CSEMA5F31C6 [9]). Entre os recursos da placa relevantes ao projeto, destacam-se os 64 MiB de SDRAM externa de 16 bits, destinada exclusivamente a abrigar a ROM de cartucho (PAK_ROM), região que, na fase final do projeto, poderá ser *escrita* para carregar uma ROM a partir de um cartão SD, e os blocos M10K de memória *on-chip*, que hospedam as demais regiões internas do mapa (BIOS, EWRAM, IWRAM, Palette RAM, VRAM e OAM). A saída VGA, com DAC de 8 bits por canal, serve à interface gráfica, enquanto os quatro *push-buttons*, os dez *switches*, os LEDs e os seis *displays* de sete segmentos são empregados na depuração e na injeção de estímulos, por exemplo, KEY[0] como *reset*, KEY[1..3] como nIRQ/nFIQ/Abort e SW[9] como *trigger* do SignalTap. Por fim, a PLL interna deriva a frequência inicial de operação do núcleo, com divisão programável que permite reduzi-la a níveis convenientes para a depuração visual.

3.2 Fluxo de Projeto e Ferramentas

O fluxo de desenvolvimento articula um conjunto de ferramentas consolidadas. A síntese, o *place-and-route*, a análise temporal e a geração do *bitstream* cabem ao Quartus Prime 15.1 [9],

enquanto a simulação RTL funcional é conduzida no ModelSim/QuestaSim por meio de *scripts* TCL dedicados a cada cenário de teste. Os *cores* de PLL, de controle de *clock* e de SignalTap são gerados no Qsys/Platform Designer. A geração dos programas de teste, em ARM e Thumb, parte de código *assembly* processado pela toolchain arm-none-eabi (as, ld e objcopy) e posteriormente convertido em formato MIF para a inicialização das memórias *on-chip*. A verificação em hardware, por sua vez, apoia-se no SignalTap, com *taps* sobre os registradores r_0 – r_{15} e sobre os barramentos de endereços e de dados.

3.3 Descrição da Prova de Conceito

A Prova de Conceito consiste na execução, na placa DE1-SoC, de programas binários compilados pela toolchain ARM, com observação direta do estado do processador via SignalTap e dos *displays* HEX e, na etapa final do projeto, com exibição em monitor VGA. Para fins de organização, o sistema é concebido em quatro camadas, núcleo, memória, vídeo e integração, que avançam de modo incremental; a Figura 4 sintetiza a *arquitetura-objetivo* do projeto, destacando, por meio de cores, aquilo que já se encontra implementado e validado, o que está em desenvolvimento e o que permanece planejado. **No estágio correspondente a esta entrega intermediária, a primeira camada já está concluída e validada, e é ela que constitui a demonstração ora apresentada;** as demais encontram-se em desenvolvimento ou planejadas, conforme detalhado a seguir.

3.3.1 Camada 1: Núcleo ARM7TDMI (implementada e validada)

A camada mais interna corresponde à descrição RTL do núcleo, organizada em módulos Verilog independentes, arm7tdmi_top, decoder, reg_bank, alu, b_shifter, multiplier, incrementer, address_reg e write_data_reg. O decoder concentra a máquina de estados multi-ciclo do processador, controlando os sinais de habilitação e seleção dos demais blocos e conduzindo o tratamento de exceções (vetores de Reset, Undefined, SWI, Prefetch Abort, Data Abort, IRQ e FIQ), ao passo que o reg_bank implementa os bancos de registradores espelhados por modo, incluindo o banco completo de FIQ. Esta camada já se encontra implementada e validada por meio de programas de teste em ARM (instrucoes.s), Thumb (arm7tdmi_thumb_test.s) e de exceções (interrupt_test.s), tanto em simulação no QuestaSim quanto em execução na placa DE1-SoC. É justamente esse núcleo em funcionamento — capaz de buscar, decodificar e executar instruções ARM e Thumb e de responder a interrupções e exceções, com o estado interno observável via SignalTap e na saída HEX, que materializa a

Prova de Conceito apresentada nesta entrega intermediária. A Figura 3 apresenta a *netlist* RTL do projeto no estágio atual, gerada pela ferramenta de síntese, na qual já coexistem o núcleo ARM7TDMI validado e os blocos do subsistema de memória em integração.

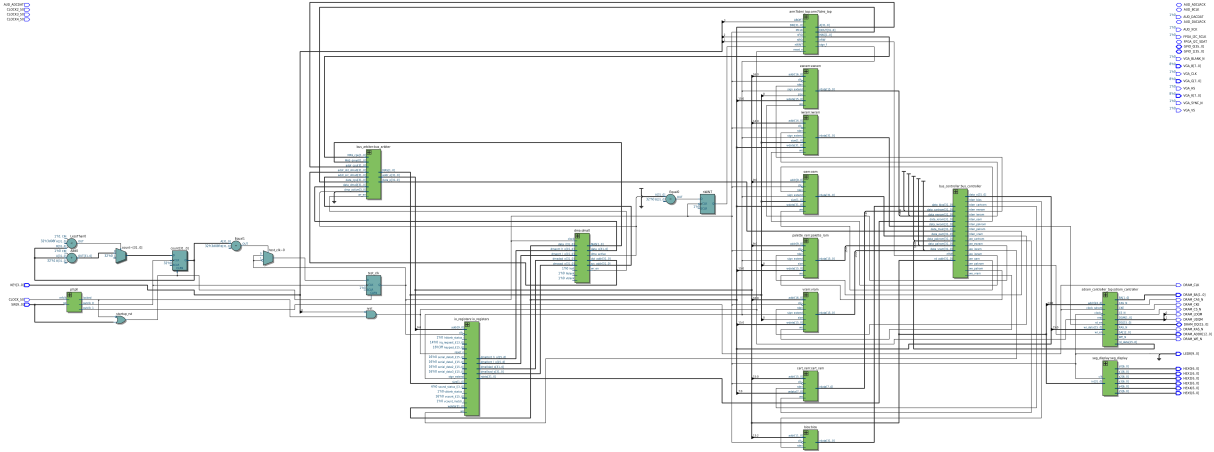


Figura 3: *Netlist* RTL do projeto no estágio atual, extraída pelo *RTL Viewer* do Quartus Prime após a análise e síntese. Reúne o núcleo ARM7TDMI já implementado e validado e os blocos do subsistema de memória ainda em integração (*bus_controller*, módulos de região e controlador de SDRAM). Fonte: autoria própria.

3.3.2 Camada 2: Subsistema de Memória (integrada, em validação)

A camada de memória encontra-se descrita e *integrada* ao *datapath* do sistema, embora a validação funcional do caminho integrado ainda esteja em andamento. Nesta entrega, o mapa de memória do GBA, e não mais a SRAM plana provisória que serviu à validação da Camada 1, constitui o caminho de memória ativo no projeto. O módulo *sram.v*, que descreve uma SRAM de 4 KiB inferida em blocos M10K (com suporte a *byte enables*, alinhamento e extensão de sinal) e serviu de memória unificada durante a validação do núcleo, permanece no repositório apenas como referência, desativado no topo em favor do mapa completo.

A decodificação do mapa cabe ao *bus_controller.v*, que interpreta o *nibble* alto do endereço (*addr[27:24]*), gera os sinais de *read/write enable* por região e multiplexa os dados lidos de volta ao processador. Acima dele, o *bus_arbiter.v* arbitra o acesso ao barramento único entre o processador e os quatro canais de DMA, ao passo que o módulo *dma.v* (canal DMA0, no momento) executa as transferências em bloco a partir dos registradores de controle providos pelo *io_registers.v*. Os módulos de região (*bios.v*, *ewram.v*, *iwram.v*, *io_registers.v*, *palette_ram.v*, *vram.v*, *oam.v* e *cart_ram.v*), inferidos em blocos M10K *on-chip*, modelam cada faixa interna do mapa, com tratamento de tamanho de acesso e extensão de sinal por região.

A ROM de cartucho (PAK_ROM) é a única região mapeada na SDRAM externa da DE1-SoC, servida por um controlador de SDRAM próprio (`sdr_controller.v` e seu envoltório `sdr_controller_top.v`), já instanciado no topo do projeto. Esse controlador, desenvolvido para substituir a IP de referência antes utilizada, opera o núcleo de acesso à SDRAM em um domínio de *clock* próprio e realiza a sincronização entre esse domínio e o domínio do processador por meio de uma lógica de *clock-domain crossing* com retenção de requisição, de modo que uma transação que coincida com um ciclo de *refresh* não seja perdida. Como a região é escrevível, prevê-se, na fase final do projeto, utilizá-la para carregar uma ROM a partir de um cartão SD para a SDRAM.

3.3.3 Camada 3: Saída de Vídeo (planejada)

A camada de vídeo, ainda não implementada no momento desta entrega, compreenderá um gerador VGA com sincronismos horizontal e vertical para uma resolução padrão (a definir entre 640×480 e 800×600 , ambas a 60 Hz), o mapeamento do *framebuffer* do GBA (240×160) sobre a região visível do monitor com escala inteira e o suporte inicial ao Modo 3 do console (*bitmap* 16 bpp), o mais simples de implementar e suficiente para demonstrar a saída de vídeo.

3.3.4 Camada 4: Integração e Validação (planejada)

A camada de integração reunirá as três anteriores em uma arquitetura única, instanciando o `arm7tdmi_top` sobre o `bus_controller` e este sobre as memórias e periféricos. Sua validação se dará pela execução de programas de teste em *assembly* cobrindo categorias específicas de instruções, estratégia já adotada na Camada 1, pela execução de uma ROM *homebrew* de baixo porte como demonstração final, com saída de vídeo no monitor VGA, e pela comparação cruzada do comportamento observado em FPGA com o do emulador NanoBoyAdvance [13] executando a mesma ROM, tomado como referência funcional.

3.4 Metodologia de Verificação

A verificação apoia-se em três níveis complementares. O primeiro é a simulação RTL no QuestaSim, dirigida por *scripts* TCL (`test.tcl` e `interrupt_test.tcl`), com inspeção das formas de onda dos barramentos, dos registradores do `reg_bank`, do *datapath* interno do decoder e dos sinais de controle. O segundo é a execução em hardware na DE1-SoC, com captura, via SignalTap, dos registradores r_0-r_{15} e dos barramentos de endereços e de dados. O terceiro é a comparação com uma referência externa: para cada cenário, a sequência de estados

esperada é obtida executando-se o mesmo binário em um emulador de referência [13, 14], que assim cumpre o papel de *golden model*. Cabe observar que, nesta entrega intermediária, os dois primeiros níveis já se encontram em uso efetivo sobre o núcleo ARM7TDMI, ao passo que a comparação sistemática contra o *golden model* será consolidada à medida que as camadas de memória e vídeo forem integradas.

3.5 Cronograma Macro

O Quadro 1 apresenta o cronograma macro de execução do projeto. A presente entrega intermediária corresponde à conclusão das fases F1 e F2, estudo da arquitetura e implementação RTL do núcleo ARM7TDMI, com sua validação em simulação e em hardware, e ao andamento da fase F3, na qual o subsistema de memória já foi descrito e integrado ao *datapath*, restando a validação funcional do caminho integrado. O projeto encontra-se, neste momento, na fase F3; as fases F4 a F6 descrevem, portanto, a continuidade prevista até a entrega final. O cronograma detalhado, com desdobramento semanal e marcos intermediários, constitui entrega complementar e está localizado em `documentos_de_entrega/Cronograma/`.

Tabela 1: Cronograma macro de execução.

Fase	Mês	Atividades
F1	1 e 2	Estudo da arquitetura ARM7TDMI e do mapa de memória do GBA. (<i>concluída</i>)
F2	3 e 4	Implementação RTL do núcleo ARM7TDMI; Síntese física do núcleo na DE1-SoC; programas de teste em ARM e Thumb; validação por SignalTap; tratamento completo de exceções e IRQ/FIQ. (<i>concluída, entrega intermediária</i>)
F3	5 e 6	Subsistema de memória: <code>bus_controller</code> , <code>bus_arbiter</code> , DMA, módulos de região, controlador de SDRAM; integração ao núcleo concluída, validação funcional do caminho integrado em andamento. (<i>em andamento</i>)
F4	7 e 8	Implementação do PPU e produzir imagem em um display. (<i>previsto</i>)
F5	9 e 10	Implementação do comando do usuário e gerador de som. (<i>previsto</i>)
F6	11 e 12	Validação com um jogo real: transferência de uma ROM de um SD Card para SDRAM. (<i>previsto</i>)

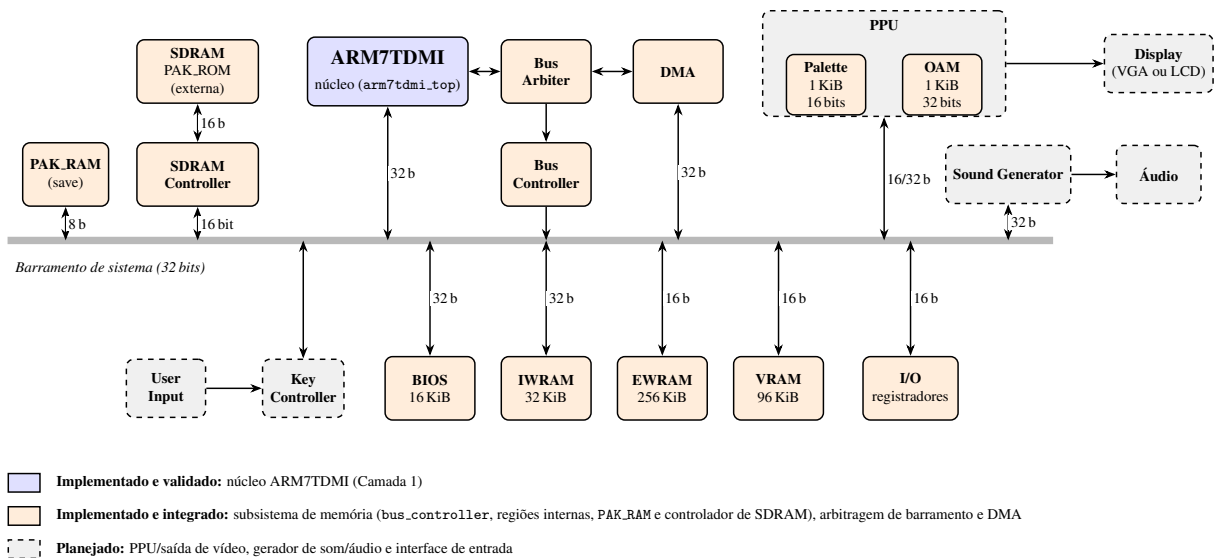


Figura 4: Arquitetura-objetivo do projeto. O núcleo ARM7TDMI (azul) já se encontra implementado e validado; o subsistema de memória (laranja), bus_controller, regiões internas (BIOS, EWRAM, IWRAM, VRAM, I/O, Palette, OAM), PAK_RAM e o controlador da SDRAM externa, que abriga exclusivamente a PAK_ROM, além da arbitragem de barramento e do DMA, encontra-se implementado e integrado ao *datapath*, com a validação funcional do caminho integrado ainda em andamento; os blocos de PPU e saída de vídeo, gerador de som e áudio, e a interface de entrada (*Key Controller*) permanecem planejados (cinza). As larguras indicadas correspondem ao barramento nominal de cada região; a decodificação por região emprega o *nibble* alto do endereço (addr[27:24]).

REFERÊNCIAS

- [1] ARM LIMITED. **ARM7TDMI Technical Reference Manual**. Cambridge: ARM Limited, 2004. Documento DDI 0210B.
- [2] ARM LIMITED. **ARM7TDMI (Rev 3) Core Processor Product Overview**. Cambridge: ARM Limited, 2001. Documento DVI 0027B.
- [3] COPETTI, R. **Game Boy Advance Architecture: a Practical Analysis**. 2019. Disponível em: <https://www.copetti.org/writings/consoles/game-boy-advance/>. Acesso em: 15 abr. 2026.
- [4] VIJN, J. **Tonc, GBA Programming in Rot13**. Disponível em: <https://gbadev.net/tonc/hardware.html>. Acesso em: 15 abr. 2026.
- [5] KORTH, M. **GBATEK, GBA/NDS Technical Information**. Disponível em: <https://problemkaputt.de/gbatek.htm>. Acesso em: 15 abr. 2026.
- [6] PATTERSON, D. A.; HENNESSY, J. L. **Computer Organization and Design: The Hardware/Software Interface**. 6. ed. Burlington: Morgan Kaufmann, 2020.
- [7] HARRIS, D. M.; HARRIS, S. L. **Digital Design and Computer Architecture, ARM Edition**. Burlington: Morgan Kaufmann, 2015.
- [8] TRIMBERGER, S. M. Three Ages of FPGAs: A Retrospective on the First Thirty Years of FPGA Technology. **Proceedings of the IEEE**, v. 103, n. 3, p. 318–331, 2015.
- [9] INTEL CORPORATION (ALTERA). **Cyclone V Device Handbook**. San Jose: Intel Corporation, 2018.
- [10] TERIC INC. **DE1-SoC User Manual**. Hsinchu: Teric Inc., 2018.
- [11] MiSTer FPGA PROJECT. **MiSTer FPGA Project, Open-Source Hardware Reproduction of Classic Consoles**. Disponível em: https://github.com/MiSTer-devel/Main_MiSTer/wiki. Acesso em: 15 abr. 2026.
- [12] ANALOGUE INC. **Analogue Pocket: Technical Specifications**. Disponível em: <https://www.analogue.co/pocket>. Acesso em: 15 abr. 2026.
- [13] SCHIBLI, F. **NanoBoyAdvance: a Cycle-Accurate Game Boy Advance Emulator**. Disponível em: <https://github.com/nba-emu/NanoBoyAdvance>. Acesso em: 15 abr. 2026.
- [14] ENDRIFT. **mGBA: a Game Boy Advance Emulator**. Disponível em: <https://mgba.io/>. Acesso em: 15 abr. 2026.